

WEEK 3

NAME – AKSHARA BHARGAVA

EMAIL ID – aksharabhargava6@gmail.com

Exercise 1: Creating Microservices for Account and Loan

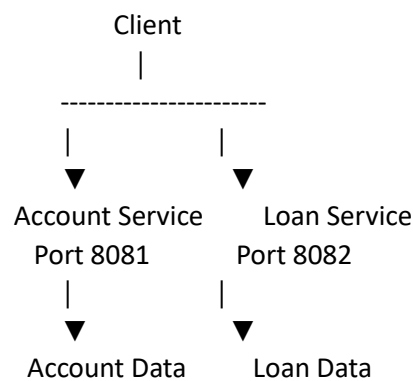
Objective

Create two independent Spring Boot microservices:

- **Account Service** (Port: **8081**)
- **Loan Service** (Port: **8082**)

Each service exposes its own REST API.

Microservice Architecture



Each microservice:

- Runs independently.
 - Has its own controller.
 - Has its own port.
 - Can be deployed separately.
-

PART 1 – Account Service

Project Structure

```
AccountService
|
├── src/main/java
|   └── com.cognizant.account
|       ├── AccountServiceApplication.java
|       ├── controller
|       │   └── AccountController.java
|       └── model
```

```
|       Account.java
|
└─ application.properties
```

Step 1: pom.xml

```
<dependencies>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

</dependencies>
```

Step 2: Account.java

```
package com.cognizant.account.model;

public class Account {

    private int accountNumber;
    private String accountHolder;
    private double balance;

    public Account() {
    }

    public Account(int accountNumber, String accountHolder, double balance) {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = balance;
    }

    public int getAccountNumber() {
        return accountNumber;
    }

    public void setAccountNumber(int accountNumber) {
        this.accountNumber = accountNumber;
    }

    public String getAccountHolder() {
        return accountHolder;
    }
}
```

```
public void setAccountHolder(String accountHolder) {
    this.accountHolder = accountHolder;
}

public double getBalance() {
    return balance;
}

public void setBalance(double balance) {
    this.balance = balance;
}
}
```

Step 3: AccountController.java

```
package com.cognizant.account.controller;

import com.cognizant.account.model.Account;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class AccountController {

    @GetMapping("/account")
    public Account getAccount() {

        return new Account(
            101,
            "Harshpreet",
            75000
        );

    }

}
```

Step 4: AccountServiceApplication.java

```
package com.cognizant.account;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class AccountServiceApplication {
```

```
public static void main(String[] args) {  
    SpringApplication.run(AccountServiceApplication.class, args);  
}  
}
```

Step 5: application.properties

```
server.port=8081
```

Test

GET http://localhost:8081/account

Response

```
{  
  "accountNumber":101,  
  "accountHolder":"Harshpreet",  
  "balance":75000  
}
```

PART 2 – Loan Service

Project Structure

```
LoanService  
|  
├── src/main/java  
|   └── com.cognizant.loan  
|       ├── LoanServiceApplication.java  
|       ├── controller  
|       |   └── LoanController.java  
|       └── model  
|           └── Loan.java  
└── application.properties
```

Loan.java

```
package com.cognizant.loan.model;
```

```
public class Loan {  
  
    private int loanId;
```

```

private String loanType;
private double amount;

public Loan() {
}

public Loan(int loanId, String loanType, double amount) {
    this.loanId = loanId;
    this.loanType = loanType;
    this.amount = amount;
}

public int getLoanId() {
    return loanId;
}

public void setLoanId(int loanId) {
    this.loanId = loanId;
}

public String getLoanType() {
    return loanType;
}

public void setLoanType(String loanType) {
    this.loanType = loanType;
}

public double getAmount() {
    return amount;
}

public void setAmount(double amount) {
    this.amount = amount;
}
}

```

LoanController.java

```

package com.cognizant.loan.controller;

import com.cognizant.loan.model.Loan;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class LoanController {

```

```
@GetMapping("/loan")
public Loan getLoan() {

    return new Loan(
        201,
        "Home Loan",
        2500000
    );

}

}
```

LoanServiceApplication.java

```
package com.cognizant.loan;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LoanServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(LoanServiceApplication.class, args);
    }

}
```

application.properties

```
server.port=8082
```

Test

GET http://localhost:8082/loan

Response

```
{
  "loanId":201,
  "loanType":"Home Loan",
  "amount":2500000
}
```

Running Both Services

Service	Port	URL
Account Service	8081	http://localhost:8081/account
Loan Service	8082	http://localhost:8082/loan

Both applications run simultaneously because they use different ports.

Exercise 2: Creating an API Gateway

Objective

Create an **API Gateway** using **Spring Cloud Gateway** to route client requests to different microservices.

Instead of accessing services directly:

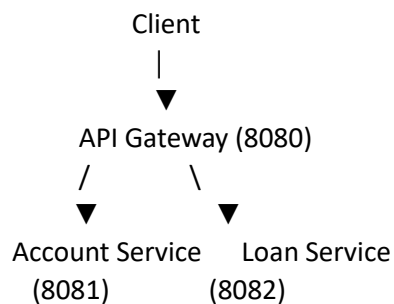
http://localhost:8081/account
http://localhost:8082/loan

The client accesses only the Gateway:

http://localhost:8080/account
http://localhost:8080/loan

The Gateway forwards the request to the appropriate microservice.

Architecture



Project Structure

```
ApiGateway
|
|-- src/main/java
|   |-- com.cognizant.gateway
|       GatewayApplication.java
|
|-- src/main/resources
|   application.yml
```

|
└─ pom.xml

Step 1: pom.xml

<dependencies>

```
<!-- Spring Cloud Gateway -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
```

</dependencies>

<dependencyManagement>

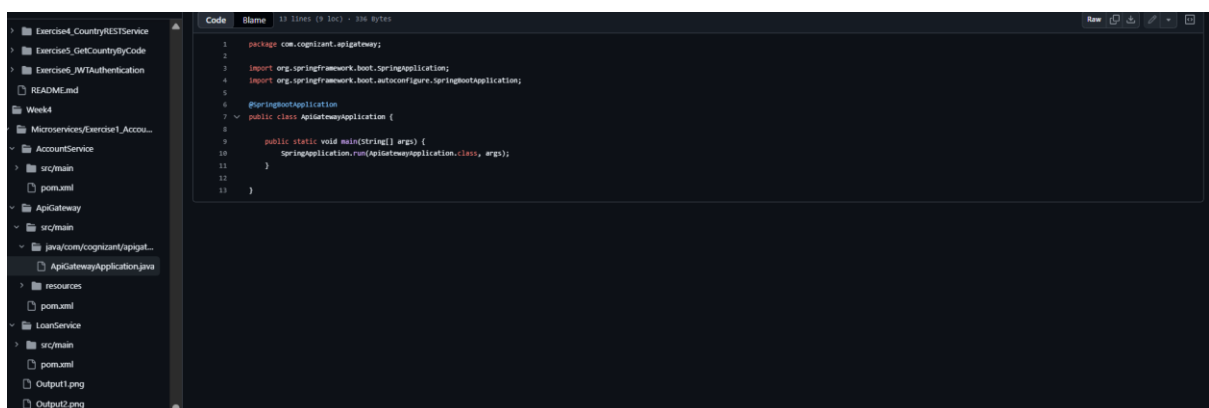
```
<dependencies>

  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-dependencies</artifactId>
    <version>2023.0.2</version>
    <type>pom</type>
    <scope>import</scope>
  </dependency>
```

</dependencies>

</dependencyManagement>

Step 2: GatewayApplication.java



package com.cognizant.gateway;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;


```
@SpringBootApplication
public class GatewayApplication {

    public static void main(String[] args) {
        SpringApplication.run(GatewayApplication.class, args);
    }

}
```

Step 3: application.yml

```
server:
  port: 8080

spring:
  cloud:
    gateway:
      routes:

        - id: account-service
          uri: http://localhost:8081
          predicates:
            - Path=/account/**

        - id: loan-service
          uri: http://localhost:8082
          predicates:
            - Path=/loan/**
```

Start the Applications

Run them in this order:

1. Account Service → Port **8081**
 2. Loan Service → Port **8082**
 3. API Gateway → Port **8080**
-

Testing

Request 1

GET <http://localhost:8080/account>

Gateway forwards the request to:

http://localhost:8081/account

Response

```
{
  "accountNumber":101,
  "accountHolder":"Harshpreet",
  "balance":75000
}
```

Request 2

GET http://localhost:8080/loan

Gateway forwards the request to:

http://localhost:8082/loan

Response

```
{
  "loanId":201,
  "loanType":"Home Loan",
  "amount":2500000
}
```

How Routing Works

Client

|

GET /account

|



Gateway

|

Matches Path=/account/**

|



localhost:8081/account

Exercise 3: Service Registration and Discovery (Eureka Server)

Objective

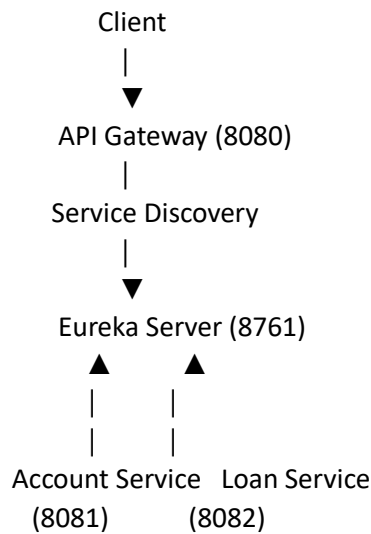
Instead of using **hardcoded URLs** in the API Gateway, register all microservices with **Netflix Eureka Server**.

Benefits:

- Automatic service registration
- Dynamic service discovery

- No need to remember service ports

Architecture



Step 1: Create Eureka Server

Project Structure

```
EurekaServer
|
|-- src/main/java
|   |-- com.cognizant.eureka
|       |-- EurekaServerApplication.java
|
|-- src/main/resources
|   |-- application.properties
|
|-- pom.xml
```

pom.xml

```
<dependencies>

    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
    </dependency>

</dependencies>

<dependencyManagement>
```

```
<dependencies>

    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-dependencies</artifactId>
        <version>2023.0.2</version>
        <type>pom</type>
        <scope>import</scope>
    </dependency>

</dependencies>
```

```
</dependencyManagement>
```

EurekaServerApplication.java

```
package com.cognizant.eureka;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }

}
```

application.properties

```
server.port=8761

spring.application.name=EUREKA-SERVER

eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
```

Run the application.

Open:

http://localhost:8761

Initially, no services will be registered.

Step 2: Register Account Service

Add Dependency

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

application.properties

spring.application.name=ACCOUNT-SERVICE

server.port=8081

eureka.client.service-url.defaultZone=http://localhost:8761/eureka

Step 3: Register Loan Service

application.properties

spring.application.name=LOAN-SERVICE

server.port=8082

eureka.client.service-url.defaultZone=http://localhost:8761/eureka

Step 4: Configure API Gateway

Instead of:

uri: http://localhost:8081

Use:

uri: lb://ACCOUNT-SERVICE

and

uri: lb://LOAN-SERVICE

application.yml

server:

port: 8080

spring:

application:

name: API-GATEWAY

cloud:

gateway:

routes:

- id: account-service

- uri: lb://ACCOUNT-SERVICE

- predicates:

- Path=/account/**

- id: loan-service

- uri: lb://LOAN-SERVICE

- predicates:

- Path=/loan/**

eureka:

client:

service-url:

defaultZone: http://localhost:8761/eureka

The lb:// prefix tells the gateway to use **load balancing** with Eureka instead of a hardcoded URL.

Running Order

Start the applications in this order:

1. Eureka Server (8761)
 2. Account Service (8081)
 3. Loan Service (8082)
 4. API Gateway (8080)
-

Eureka Dashboard

Open:

<http://localhost:8761>

You should see:

Instances currently registered with Eureka

ACCOUNT-SERVICE

LOAN-SERVICE

API-GATEWAY

Testing

Request

GET http://localhost:8080/account

Gateway flow:

Gateway

|



Eureka Server

|

Find ACCOUNT-SERVICE

|



Account Service

Request

GET http://localhost:8080/loan

Flow:

Gateway

|



Eureka Server

|

Find LOAN-SERVICE

|



Loan Service

How Eureka Works

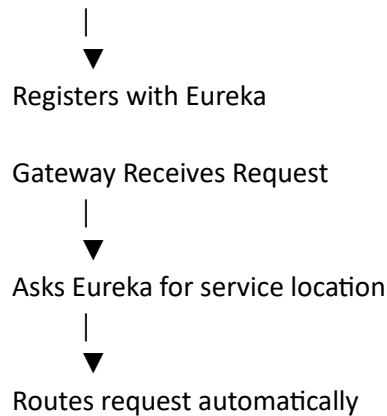
Account Service Starts

|



Registers with Eureka

Loan Service Starts



Exercise 4 (Line 28): Implementing JWT Authentication

Objective

Build a Spring Boot application that:

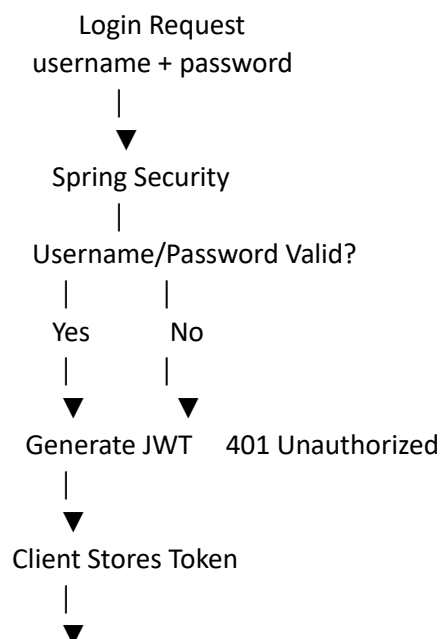
- Authenticates a user.
- Generates a **JWT (JSON Web Token)** after successful login.
- Uses the JWT to access protected REST APIs.

What is JWT?

JWT (JSON Web Token) is a secure token used for authentication between a client and a server.

Instead of storing sessions on the server, the server sends a signed token to the client. The client includes this token in future requests.

JWT Authentication Flow



Future Requests with Token
|
Authorization: Bearer <JWT>
|
▼
Protected REST API

Project Structure

```
JwtAuthentication
|
|— src/main/java
|   |— com.cognizant.jwt
|       |— JwtApplication.java
|       |— config
|           | SecurityConfig.java
|       |— controller
|           | AuthController.java
|       |— dto
|           | AuthRequest.java
|           | AuthResponse.java
|       |— service
|           | JwtService.java
|       |— util
|           | JwtUtil.java
|
|— application.properties
```

Step 1: pom.xml

```
<dependencies>

    <!-- Spring Boot Web -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Spring Security -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <!-- JWT -->
    <dependency>
```

```
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-api</artifactId>
<version>0.11.5</version>
</dependency>

<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-impl</artifactId>
<version>0.11.5</version>
<scope>runtime</scope>
</dependency>

<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-jackson</artifactId>
<version>0.11.5</version>
<scope>runtime</scope>
</dependency>

</dependencies>
```

Step 2: AuthRequest.java

```
package com.cognizant.jwt.dto;

public class AuthRequest {

    private String username;
    private String password;

    public AuthRequest() {}

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

```
}  
}
```

Step 3: AuthResponse.java

```
package com.cognizant.jwt.dto;  
  
public class AuthResponse {  
  
    private String token;  
  
    public AuthResponse(String token) {  
        this.token = token;  
    }  
  
    public String getToken() {  
        return token;  
    }  
}
```

Step 4: JwtUtil.java

```
package com.cognizant.jwt.util;  
  
import io.jsonwebtoken.Jwts;  
import io.jsonwebtoken.SignatureAlgorithm;  
  
import java.util.Date;  
  
public class JwtUtil {  
  
    private static final String SECRET_KEY = "mysecretkey";  
  
    public static String generateToken(String username) {  
  
        return Jwts.builder()  
            .setSubject(username)  
            .setIssuedAt(new Date())  
            .setExpiration(new Date(System.currentTimeMillis() + 600000))  
            .signWith(SignatureAlgorithm.HS256, SECRET_KEY)  
            .compact();  
    }  
}
```

Step 5: JwtService.java

```

package com.cognizant.jwt.service;

import com.cognizant.jwt.util.JwtUtil;
import org.springframework.stereotype.Service;

@Service
public class JwtService {

    public String authenticate(String username, String password) {

        if ("admin".equals(username)
            && "admin123".equals(password)) {

            return JwtUtil.generateToken(username);

        }

        return null;
    }
}

```

Step 6: AuthController.java

```

package com.cognizant.jwt.controller;

import com.cognizant.jwt.dto.AuthRequest;
import com.cognizant.jwt.dto.AuthResponse;
import com.cognizant.jwt.service.JwtService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/auth")
public class AuthController {

    @Autowired
    JwtService service;

    @PostMapping("/login")
    public AuthResponse login(@RequestBody AuthRequest request) {

        String token = service.authenticate(
            request.getUsername(),
            request.getPassword());

        if (token != null) {

```

```
        return new AuthResponse(token);
    }

    throw new RuntimeException("Invalid Credentials");
}
}
```

Step 7: SecurityConfig.java

```
package com.cognizant.jwt.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http)
        throws Exception {

        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth ->
                auth.anyRequest().permitAll());

        return http.build();
    }
}
```

Step 8: JwtApplication.java

```
package com.cognizant.jwt;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class JwtApplication {

    public static void main(String[] args) {
        SpringApplication.run(JwtApplication.class, args);
    }
}
```

Testing in Postman

Login Request

POST http://localhost:8080/auth/login

Body:

```
{
  "username": "admin",
  "password": "admin123"
}
```

Response

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9. ...."
}
```

The token returned is your **JWT**.

Exercise 5 (Line 29): Securing REST APIs with Spring Security

Objective

Secure REST APIs using **Spring Security**.

We will create:

- A **public endpoint** (accessible without login)
- A **protected endpoint** (requires authentication)

Project Structure

SpringSecurityDemo

```
|
|  └─ src/main/java
|      └─ com.cognizant.security
|          └─ SecurityApplication.java
|              └─ config
|                  SecurityConfig.java
|              └─ controller
|                  EmployeeController.java
|
└─ pom.xml
```

Step 1: pom.xml

```
<dependencies>

    <!-- Spring Boot Web -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Spring Security -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

</dependencies>
```

Step 2: EmployeeController.java

```
package com.cognizant.security.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class EmployeeController {

    @GetMapping("/public")
    public String publicApi() {
        return "Welcome! This is a Public API.";
    }

    @GetMapping("/private")
    public String privateApi() {
        return "Welcome! This is a Protected API.";
    }
}
```

Step 3: SecurityConfig.java

```
package com.cognizant.security.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import org.springframework.security.config.Customizer;
```

```
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
```

```
import org.springframework.security.core.userdetails.User;
```

```
import org.springframework.security.core.userdetails.UserDetails;
```

```
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
```

```
import org.springframework.security.core.userdetails.UserDetailsService;
```

```
import org.springframework.security.web.SecurityFilterChain;
```

```
@Configuration
```

```
public class SecurityConfig {
```

```
    @Bean
```

```
    public UserDetailsService users() {
```

```
        UserDetails user =
```

```
            User.withDefaultPasswordEncoder()
```

```
                .username("admin")
```

```
                .password("admin123")
```

```
                .roles("ADMIN")
```

```
                .build();
```

```
        return new InMemoryUserDetailsManager(user);
```

```
    }
```

```
    @Bean
```

```
    public SecurityFilterChain filterChain(HttpSecurity http)
```

```
        throws Exception {
```

```
        http
```

```
            .csrf(csrf -> csrf.disable())
```

```
            .authorizeHttpRequests(auth -> auth
```

```
                .requestMatchers("/public").permitAll()
```

```
                .anyRequest().authenticated()
```

```
            )
```

```
            .httpBasic(Customizer.withDefaults());
```

```
        return http.build();
```

```
    }
```

```
}
```

Step 4: SecurityApplication.java

```
package com.cognizant.security;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SecurityApplication {

    public static void main(String[] args) {
        SpringApplication.run(SecurityApplication.class, args);
    }

}
```

Testing in Postman

Public API

Request

GET http://localhost:8080/public

Response

Welcome! This is a Public API.

No login is required.

Protected API

Request

GET http://localhost:8080/private

Without credentials:

401 Unauthorized

Login Credentials

Username : admin

Password : admin123

In Postman:

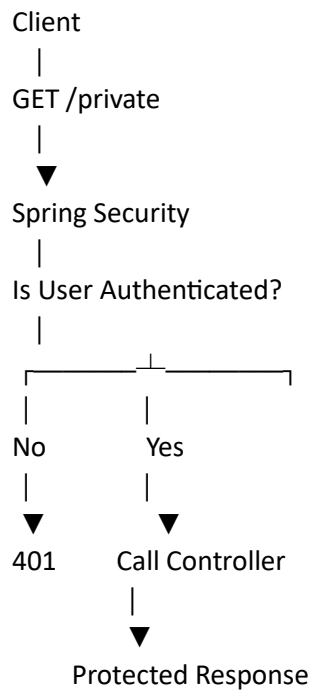
- Authorization
- Type → Basic Auth
- Username → admin

- Password → admin123

Response

Welcome! This is a Protected API.

Security Flow



Important Classes

Class	Purpose
SecurityFilterChain	Configures security rules
UserDetailsService	Provides user information
InMemoryUserDetailsManager	Stores users in memory
HttpSecurity	Configures authentication and authorization

Authentication vs Authorization

Authentication	Authorization
Verifies who the user is	Determines what the user can access

Authentication

Login

Username & Password

Authorization

Access control

Roles and Permissions

Week 3 – Spring Boot 3 + JWT + Spring Security

Exercise 6 (Line 30): Role-Based Authorization

Objective

Implement **Role-Based Authorization** using Spring Security.

Create two users:

- **ADMIN**
- **USER**

Restrict access to REST APIs based on user roles.

Project Structure

RoleBasedSecurity

```
|
|— src/main/java
|   └─ com.cognizant.rolebased
|       ├── RoleBasedApplication.java
|       ├── config
|       │   └─ SecurityConfig.java
|       └─ controller
|           └─ HomeController.java
└─ pom.xml
```

Step 1: pom.xml

<dependencies>

<!-- Spring Boot Web -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-web</artifactId>

</dependency>

<!-- Spring Security -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-security</artifactId>

</dependency>

</dependencies>

Step 2: HomeController.java

```
package com.cognizant.rolebased.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HomeController {

    @GetMapping("/")
    public String home() {
        return "Welcome to Spring Security";
    }

    @GetMapping("/user")
    public String user() {
        return "Welcome USER!";
    }

    @GetMapping("/admin")
    public String admin() {
        return "Welcome ADMIN!";
    }

}
```

Step 3: SecurityConfig.java

```
package com.cognizant.rolebased.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;

import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;

import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.core.userdetails.UserDetailsService;
```

```

import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public UserDetailsService userDetailsService() {

        UserDetails admin = User.withDefaultPasswordEncoder()
            .username("admin")
            .password("admin123")
            .roles("ADMIN")
            .build();

        UserDetails user = User.withDefaultPasswordEncoder()
            .username("user")
            .password("user123")
            .roles("USER")
            .build();

        return new InMemoryUserDetailsManager(admin, user);
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http
            .csrf(csrf -> csrf.disable())

            .authorizeHttpRequests(auth -> auth

                .requestMatchers("/").permitAll()

                .requestMatchers("/admin").hasRole("ADMIN")

                .requestMatchers("/user")
                .hasAnyRole("USER", "ADMIN")

                .anyRequest()
                .authenticated()
            )

            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}

```

```
}
```

Step 4: RoleBasedApplication.java

```
package com.cognizant.rolebased;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class RoleBasedApplication {

    public static void main(String[] args) {
        SpringApplication.run(RoleBasedApplication.class, args);
    }

}
```

Testing in Postman

1. Home Page

GET http://localhost:8080/

Output

Welcome to Spring Security

No login required.

2. User Login

Credentials

Username : user

Password : user123

Request

GET http://localhost:8080/user

Output

Welcome USER!

3. User Accessing Admin API

GET http://localhost:8080/admin

Output

403 Forbidden

Because the **USER** role is not allowed to access /admin.

4. Admin Login

Credentials

Username : admin

Password : admin123

Request

GET http://localhost:8080/admin

Output

Welcome ADMIN!

5. Admin Accessing User API

GET http://localhost:8080/user

Output

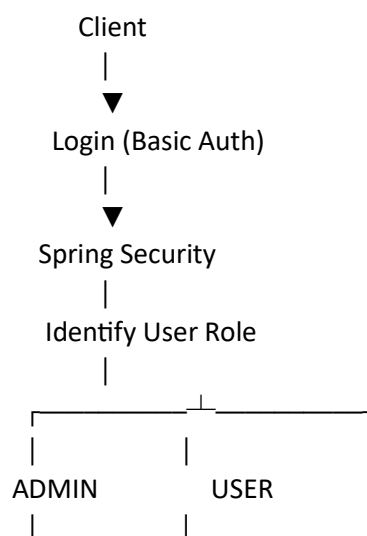
Welcome USER!

This works because:

```
.hasAnyRole("USER","ADMIN")
```

allows both roles.

Authorization Flow



▼ /admin ✓	▼ /admin ✗
/user ✓	/user ✓

Important Methods

Permit Everyone

```
.requestMatchers("/")  
.permitAll()
```

Only ADMIN

```
.requestMatchers("/admin")  
.hasRole("ADMIN")
```

USER or ADMIN

```
.requestMatchers("/user")  
.hasAnyRole("USER","ADMIN")
```

Authenticated Users

```
.anyRequest()  
.authenticated()
```